Match3D v2: 2.5D Registration Algorithm

Technical Documentation

Match3D Development Team

May 2026

Abstract

This document describes the registration algorithms implemented in Match3D v2 for aligning 2.5D depth images (heightmaps). Unlike standard 3D point cloud registration methods, our approach treats the problem as a 2D registration in the (x, y) plane with separate handling of the z (depth) axis. This design decision addresses the fundamental scale imbalance present in 2.5D data where depth values are orders of magnitude larger than lateral coordinates.

Contents

1 Introduction

1.1 The 2.5D Data Model

A 2.5D depth image (heightmap or range image) represents a surface as a regular grid of depth measurements. Each pixel at grid position (c, r) stores a single depth value $z(r, c)$. The 3D coordinates of a point are:

$$\mathbf{p}(r, c) = \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} c \cdot \Delta_x \\ r \cdot \Delta_y \\ z(r, c) \end{pmatrix} \tag{1}$$

where Δ_x and Δ_y are the pixel sizes in meters.

1.2 The Scale Imbalance Problem

For typical dental surface scans:

- Lateral pixel size: $\Delta_x, \Delta_y \approx 30\,\mu\text{m} = 3 \times 10^{-5}\,\text{m}$
- Image dimensions: $\sim 500 \times 1000$ pixels
- Lateral extent: $x_{\max} \approx 0.015\,\text{m}$, $y_{\max} \approx 0.03\,\text{m}$
- Depth values: $z \approx 800$–$1000\,\mu\text{m}$ (with arbitrary baseline offset)

This creates a **scale imbalance** where $z \gg x, y$. Standard 3D registration algorithms (ICP, Horn's method) compute rotations based on 3D Euclidean distances, causing the large z values to dominate and produce erroneous rotations.

2 Registration Pipeline Overview

The registration pipeline consists of four stages:

1. **Coarse Registration** – Initial alignment using either:
   - Center of Mass (COM) translation only
   - Landmark point pairs (2D rotation + translation + z offset)
2. **Align (4-DOF)** – Iterative 2.5D refinement optimizing α, t_x, t_y, t_z
3. **Refine (6-DOF)** – Point-to-plane refinement optimizing all 6 DOF (optional)
4. **Difference Calculation** – Compute per-pixel height differences

3 Transformation Model

3.1 4-DOF Model (Align)

For typical 2.5D data, we restrict the transformation to:

- Rotation around the z-axis only (yaw angle α)
- Translation in all three axes (t_x, t_y, t_z)

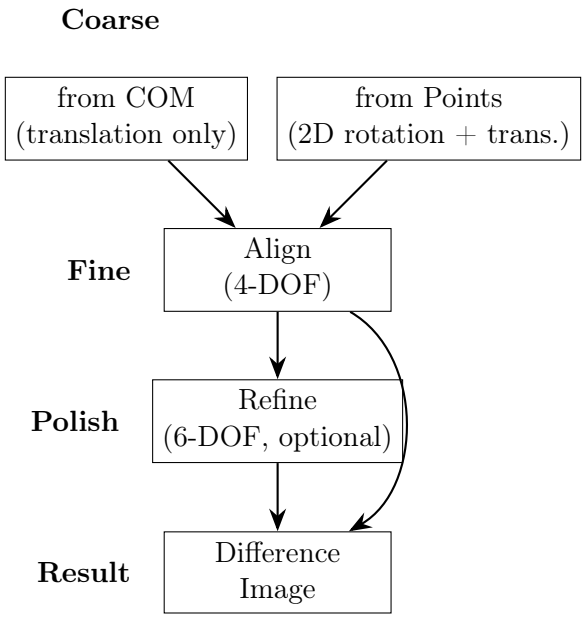

Figure 1: Registration pipeline overview

The transformation from data coordinates to model (reference) coordinates is:

$$\begin{pmatrix} x' \\ y' \\ z' \end{pmatrix} = \begin{pmatrix} \cos\alpha & -\sin\alpha & 0 \\ \sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix} + \begin{pmatrix} t_x \\ t_y \\ t_z \end{pmatrix} \tag{2}$$

Or in compact form:

$$\mathbf{p}' = \mathbf{R}_z(\alpha) \cdot \mathbf{p} + \mathbf{t} \tag{3}$$

3.2 6-DOF Model (Refine)

For the optional 6-DOF refinement, we use the full Euler angle parameterization with three rotation angles:

- α – rotation around z-axis (yaw)
- β – rotation around y-axis (pitch)
- γ – rotation around x-axis (roll)
- (t_x, t_y, t_z) – translation

For 2.5D heightmap data, β and γ should typically remain small (< 1) since the surfaces are assumed to be roughly parallel. See Section 7 for the detailed 6-DOF formulation.

4 Coarse Registration: Center of Mass (COM)

The simplest coarse alignment translates the data image so its center of mass coincides with the reference image's center of mass.

4.1 Algorithm

Given model image M and data image D with optional ROI masks:

$$\mathbf{c}_M = \frac{1}{|S_M|} \sum_{(r,c)\in S_M} \mathbf{p}_M(r,c), \quad \mathbf{c}_D = \frac{1}{|S_D|} \sum_{(r,c)\in S_D} \mathbf{p}_D(r,c) \tag{4}$$

where S_M and S_D are the sets of valid pixels within the ROI.
The translation is simply:

$$\mathbf{t} = \mathbf{c}_M - \mathbf{c}_D = \begin{pmatrix} t_x \\ t_y \\ t_z \end{pmatrix} \tag{5}$$

with $\alpha = 0$ (no rotation).

5 Coarse Registration: From Points (Landmark-Based)

When corresponding landmark points are available, we can compute both rotation and translation.

5.1 Problem Formulation

Given n corresponding point pairs:

- Data points: $\{\mathbf{d}_i\}_{i=1}^n$ with $\mathbf{d}_i = (x_i^d, y_i^d, z_i^d)^T$
- Reference points: $\{\mathbf{r}_i\}_{i=1}^n$ with $\mathbf{r}_i = (x_i^r, y_i^r, z_i^r)^T$

Find α and $\mathbf{t}$ that minimize:

$$E = \sum_{i=1}^{n} \|\mathbf{r}_i - (\mathbf{R}_z(\alpha) \cdot \mathbf{d}_i + \mathbf{t})\|^2 \tag{6}$$

5.2 2D Rotation Estimation

Key insight: For 2.5D data, we solve the rotation using only the (x, y) coordinates, ignoring z entirely.

5.2.1 Step 1: Compute 2D Centroids

$$\bar{x}^d = \frac{1}{n}\sum_{i=1}^{n} x_i^d, \quad \bar{y}^d = \frac{1}{n}\sum_{i=1}^{n} y_i^d \tag{7}$$

$$\bar{x}^r = \frac{1}{n}\sum_{i=1}^{n} x_i^r, \quad \bar{y}^r = \frac{1}{n}\sum_{i=1}^{n} y_i^r \tag{8}$$

5.2.2 Step 2: Center the Points

$$\tilde{x}_i^d = x_i^d - \bar{x}^d, \quad \tilde{y}_i^d = y_i^d - \bar{y}^d \tag{9}$$

$$\tilde{x}_i^r = x_i^r - \bar{x}^r, \quad \tilde{y}_i^r = y_i^r - \bar{y}^r \tag{10}$$

5.2.3 Step 3: Optimal 2D Rotation Angle

The optimal rotation angle that minimizes the 2D alignment error is:

$$\boxed{\alpha = \arctan 2 \left(\sum_{i=1}^{n} (\tilde{x}_i^d \tilde{y}_i^r - \tilde{y}_i^d \tilde{x}_i^r), \sum_{i=1}^{n} (\tilde{x}_i^d \tilde{x}_i^r + \tilde{y}_i^d \tilde{y}_i^r) \right)} \tag{11}$$

Derivation: This follows from the Procrustes problem in 2D. The optimal rotation minimizes:

$$E_{2D} = \sum_{i=1}^{n} \left[(\tilde{x}_i^r - \tilde{x}_i^d \cos\alpha + \tilde{y}_i^d \sin\alpha)^2 + (\tilde{y}_i^r - \tilde{x}_i^d \sin\alpha - \tilde{y}_i^d \cos\alpha)^2 \right] \tag{12}$$

Taking $\frac{\partial E_{2D}}{\partial \alpha} = 0$ yields Equation 11.

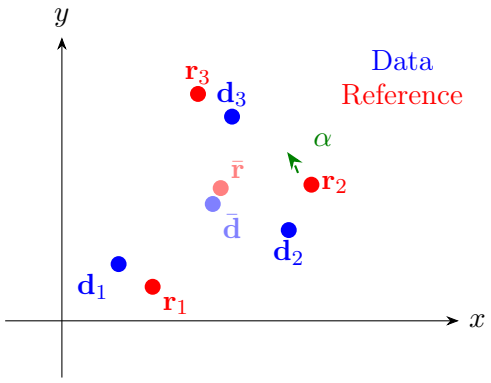

Figure 2: 2D rotation alignment: Find angle α that best aligns centered data points to reference points in the (x, y) plane.

5.3 Translation Estimation

Once α is known, the translation is computed from the centroids:

$$\begin{pmatrix} t_x \\ t_y \end{pmatrix} = \begin{pmatrix} \bar{x}^r \\ \bar{y}^r \end{pmatrix} - \begin{pmatrix} \cos\alpha & -\sin\alpha \\ \sin\alpha & \cos\alpha \end{pmatrix} \begin{pmatrix} \bar{x}^d \\ \bar{y}^d \end{pmatrix} \tag{13}$$

5.4 Z-Offset Estimation (Median)

The z-translation is computed as the **median** of the z-differences at the landmark points:

$$t_z = \text{median}\{z_i^r - z_i^d\}_{i=1}^{n} \tag{14}$$

Why median? The median is robust to outliers. If one landmark has an erroneous z-value (e.g., due to noise or a hole in the data), it won't corrupt the entire estimate.

6 Fine Registration: Align (4-DOF)

The "Align" function performs 4-DOF iterative refinement: rotation around the z-axis (α) plus translation (t_x, t_y, t_z). Unlike standard 3D ICP, our algorithm:

1. Finds correspondences based on (x, y) grid position, not 3D Euclidean distance
2. Computes only z-axis rotation (not full 3D rotation)
3. Uses median for z-offset (robust to outliers)

6.1 Algorithm

Algorithm 1 2.5D Iterative Refinement

Require: Model image M, Data image D, Initial transform $T_0 = (\alpha_0, t_x^0, t_y^0, t_z^0)$
Ensure: Refined transform $T^* = (\alpha^*, t_x^*, t_y^*, t_z^*)$

$T \leftarrow T_0$
for $k = 1$ to maxIterations **do**
  // **Step 1: Find correspondences based on (x,y) position**
  $\mathcal{C} \leftarrow \emptyset$
  for all valid model pixels (r_m, c_m) **do**
    Compute model world coordinates: $\mathbf{p}_m = (x_m, y_m, z_m)$
    Apply inverse transform to find data position: $(x_d, y_d) = T^{-1}(x_m, y_m)$
    Convert to data pixel: $(r_d, c_d) = (y_d/\Delta_y, x_d/\Delta_x)$
    if (r_d, c_d) is valid in data image **then**
      $z_d \leftarrow$ bilinear interpolation of D at (r_d, c_d)
      $\mathcal{C} \leftarrow \mathcal{C} \cup \{(\mathbf{p}_m, (x_d, y_d, z_d))\}$
    end if
  end for

  // **Step 2: Subsample if too many correspondences**
  if $|\mathcal{C}| >$ samplingLimit **then**
    $\mathcal{C} \leftarrow$ random subset of size samplingLimit
  end if

  // **Step 3: Compute 2D rotation update**
  Compute centroids $(\bar{x}_m, \bar{y}_m)$ and $(\bar{x}_d, \bar{y}_d)$ from $\mathcal{C}$
  $\Delta\alpha \leftarrow \arctan 2(\sum \text{cross}, \sum \text{dot})$ ▷ Eq. 11

  // **Step 4: Compute z-offset update**
  $\Delta t_z \leftarrow \text{median}\{z_m - z_d - t_z\}$ over $\mathcal{C}$

  // **Step 5: Compute translation update**
  Update $\alpha \leftarrow \alpha + \Delta\alpha$
  Recompute (t_x, t_y) from rotated centroids
  $t_z \leftarrow t_z + \Delta t_z$

  // **Step 6: Check convergence**
  RMS $\leftarrow \sqrt{\frac{1}{|\mathcal{C}|}\sum(z_m - z_d - t_z)^2}$
  if relative RMS decrease $<$ minRMSDecrease **then**
    break ▷ Converged
  end if
end for
return $T = (\alpha, t_x, t_y, t_z)$

6.2 Inverse Transform for Correspondence Finding

To find where a model point (x_m, y_m) comes from in the data image, we apply the inverse transform:

$$\begin{pmatrix} x_d \\ y_d \end{pmatrix} = \begin{pmatrix} \cos\alpha & \sin\alpha \\ -\sin\alpha & \cos\alpha \end{pmatrix} \begin{pmatrix} x_m - t_x \\ y_m - t_y \end{pmatrix} \tag{15}$$

Note: $\mathbf{R}_z(-\alpha) = \mathbf{R}_z(\alpha)^T$.

6.3 Bilinear Interpolation

For sub-pixel accuracy, we use bilinear interpolation to sample the data image:

$$z_d = (1 - f_r)(1 - f_c) \cdot z_{00} + (1 - f_r) f_c \cdot z_{01} + f_r (1 - f_c) \cdot z_{10} + f_r f_c \cdot z_{11} \tag{16}$$

where $f_r = r_d - \lfloor r_d \rfloor$ and $f_c = c_d - \lfloor c_d \rfloor$ are the fractional parts.

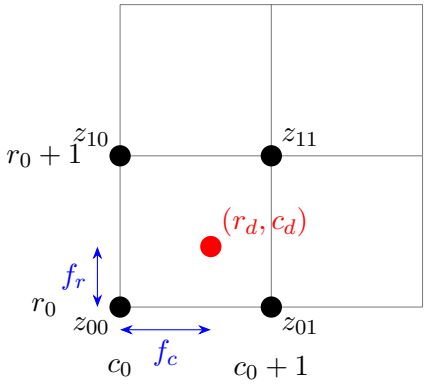

Figure 3: Bilinear interpolation: The value at (r_d, c_d) is computed as a weighted average of the four surrounding pixel values.

7 Fine Registration: Refine (6-DOF Point-to-Plane)

The "Refine" function performs full 6-DOF optimization using Neugebauer's point-to-plane ICP algorithm. This method optimizes all three Euler angles (α, β, γ) plus translation (t_x, t_y, t_z).

7.1 Motivation

While 4-DOF Align is sufficient for most 2.5D heightmap data, there are cases where the surfaces have slight tilts relative to each other (e.g., due to mounting differences between measurements). The point-to-plane algorithm provides:

- Faster convergence than point-to-point ICP (typically 3× fewer iterations)
- More accurate alignment when surfaces have local slope variations
- Ability to correct small tilts $(\beta, \gamma \neq 0)$

7.2 6-DOF Transformation Model

The full transformation uses three Euler angles in ZYX convention:

$$\mathbf{R} = \mathbf{R}_z(\alpha) \cdot \mathbf{R}_y(\beta) \cdot \mathbf{R}_x(\gamma) \tag{17}$$

where:

$$\mathbf{R}_z(\alpha) = \begin{pmatrix} \cos\alpha & -\sin\alpha & 0 \\ \sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 1 \end{pmatrix} \tag{18}$$

$$\mathbf{R}_y(\beta) = \begin{pmatrix} \cos\beta & 0 & \sin\beta \\ 0 & 1 & 0 \\ -\sin\beta & 0 & \cos\beta \end{pmatrix} \tag{19}$$

$$\mathbf{R}_x(\gamma) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos\gamma & -\sin\gamma \\ 0 & \sin\gamma & \cos\gamma \end{pmatrix} \tag{20}$$

The transformation is:

$$\mathbf{p}' = \mathbf{R} \cdot \mathbf{p} + \mathbf{t} \tag{21}$$

7.3 Point-to-Plane Distance

Given a point $\mathbf{p}_d$ on the data surface and its corresponding point $\mathbf{p}_m$ on the model surface with surface normal $\mathbf{n}_m = (n_x, n_y, n_z)^T$, the point-to-plane distance is:

$$d = \mathbf{n}_m \cdot (\mathbf{T}(\mathbf{p}_d) - \mathbf{p}_m) \tag{22}$$

This measures the **signed distance along the normal direction**, which represents how far the transformed data point is from the model's tangent plane at the corresponding point.

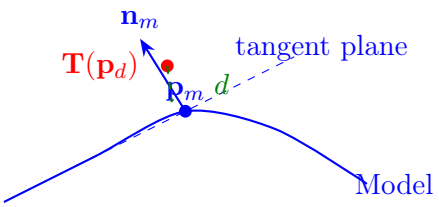

Figure 4: Point-to-plane distance: The distance d is measured along the surface normal direction, not the Euclidean distance to the point.

7.4 Surface Normal Computation

For a 2.5D heightmap, surface normals are computed from the local gradients:

$$\frac{\partial z}{\partial x} \approx \frac{z(r, c+1) - z(r, c-1)}{2\Delta_x}, \quad \frac{\partial z}{\partial y} \approx \frac{z(r+1, c) - z(r-1, c)}{2\Delta_y} \tag{23}$$

The unnormalized normal vector is:

$$\tilde{\mathbf{n}} = \begin{pmatrix} -\partial z/\partial x \\ -\partial z/\partial y \\ 1 \end{pmatrix} \tag{24}$$

Normalized:

$$\mathbf{n} = \frac{\tilde{\mathbf{n}}}{\|\tilde{\mathbf{n}}\|} = \frac{1}{\sqrt{g_x^2 + g_y^2 + 1}} \begin{pmatrix} -g_x \\ -g_y \\ 1 \end{pmatrix} \tag{25}$$

where $g_x = \partial z/\partial x$ and $g_y = \partial z/\partial y$.

7.5 Linearization for Small Angles

Following Neugebauer, we linearize the transformation for small incremental updates. For small angles $\delta\alpha, \delta\beta, \delta\gamma$:

$$\mathbf{R} \approx \mathbf{I} + \begin{pmatrix} 0 & -\delta\alpha & \delta\beta \\ \delta\alpha & 0 & -\delta\gamma \\ -\delta\beta & \delta\gamma & 0 \end{pmatrix} \tag{26}$$

A point $\mathbf{p} = (x, y, z)^T$ transforms to:

$$\mathbf{p}' \approx \begin{pmatrix} x - \delta\alpha \cdot y + \delta\beta \cdot z + \delta t_x \\ \delta\alpha \cdot x + y - \delta\gamma \cdot z + \delta t_y \\ -\delta\beta \cdot x + \delta\gamma \cdot y + z + \delta t_z \end{pmatrix} \tag{27}$$

7.6 Normal Equation System

The point-to-plane constraint for a correspondence pair becomes:

$$n_x(x' - x_m) + n_y(y' - y_m) + n_z(z' - z_m) = 0 \tag{28}$$

where (n_x, n_y, n_z) is the transformed normal and (x_m, y_m, z_m) is the model point. Substituting the linearized transformation and rearranging:

$$\mathbf{a}^T \boldsymbol{\delta} = b \tag{29}$$

where $\boldsymbol{\delta} = (\delta\alpha, \delta\beta, \delta\gamma, \delta t_x, \delta t_y, \delta t_z)^T$ and the constraint coefficients are:

$$a_0 = n_z \cdot y - n_y \cdot z \qquad \text{(rotation around } x\text{-axis)} \tag{30}$$

$$a_1 = n_x \cdot z - n_z \cdot x \qquad \text{(rotation around } y\text{-axis)} \tag{31}$$

$$a_2 = n_y \cdot x - n_x \cdot y \qquad \text{(rotation around } z\text{-axis)} \tag{32}$$

$$a_3 = n_x \qquad \text{(translation in } x\text{)} \tag{33}$$

$$a_4 = n_y \qquad \text{(translation in } y\text{)} \tag{34}$$

$$a_5 = n_z \qquad \text{(translation in } z\text{)} \tag{35}$$

$$b = n_z \cdot (z_m - z) + n_y \cdot (y_m - y) + n_x \cdot (x_m - x) \tag{36}$$

Accumulating over all correspondence pairs gives the 6×6 normal equation system:

$$\mathbf{A}^T\mathbf{A} \cdot \boldsymbol{\delta} = \mathbf{A}^T\mathbf{b} \tag{37}$$

This is solved using Cholesky decomposition (for positive definite $\mathbf{A}^T\mathbf{A}$).

7.7 Algorithm

Algorithm 2 6-DOF Point-to-Plane Refinement (Neugebauer)

Require: Model image M, Data image D, Initial transform $T_0 = (\alpha, \beta, \gamma, t_x, t_y, t_z)$
Ensure: Refined transform T^*

$T \leftarrow T_0$, prevRMS $\leftarrow \infty$
for $k = 1$ to maxIterations **do**
  $\mathbf{N} \leftarrow \mathbf{0}_{6\times6}$, $\mathbf{r} \leftarrow \mathbf{0}_6$ ▷ Normal equation accumulator
  sumSqErr $\leftarrow 0$, count $\leftarrow 0$
  for all valid model pixels (r_m, c_m) (optionally subsampled) **do**
    Compute model point $\mathbf{p}_m$ and gradients g_x, g_y
    Compute normal $\mathbf{n} = (-g_x, -g_y, 1)^T / \|...\|$
    Transform normal: $\mathbf{n}' = \mathbf{R} \cdot \mathbf{n}$
    Apply inverse transform to find data position (x_d, y_d)
    if data position valid **then**
      Interpolate z_d, compute $\mathbf{p}_d$
      Transform: $\mathbf{p}'_d = \mathbf{R} \cdot \mathbf{p}_d + \mathbf{t}$
      Compute coefficients $\mathbf{a}$ and b from linearization
      Accumulate: $\mathbf{N} \leftarrow \mathbf{N} + \mathbf{a}\mathbf{a}^T$, $\mathbf{r} \leftarrow \mathbf{r} + b \cdot \mathbf{a}$
      sumSqErr $\leftarrow$ sumSqErr $+ b^2$
      count $\leftarrow$ count $+ 1$
    end if
  end for

  Solve $\mathbf{N} \cdot \boldsymbol{\delta} = \mathbf{r}$ via Cholesky decomposition

  // **Update transform (compose incremental with current)**
  $\alpha \leftarrow \alpha + \delta_0$, $\beta \leftarrow \beta + \delta_1$, $\gamma \leftarrow \gamma + \delta_2$
  $t_x \leftarrow t_x + \delta_3$, $t_y \leftarrow t_y + \delta_4$, $t_z \leftarrow t_z + \delta_5$
  Rebuild rotation matrix $\mathbf{R}$ from updated angles

  // **Check convergence**
  RMS $\leftarrow \sqrt{\text{sumSqErr/count}}$
  if (prevRMS − RMS)/prevRMS $<$ minRMSDecrease **then**
    break
  end if
  prevRMS $\leftarrow$ RMS
end for
return $T = (\alpha, \beta, \gamma, t_x, t_y, t_z)$

7.8 Outlier Handling

Points with large residuals are excluded from the normal equation accumulation. A point is considered an outlier if:

$$|b| > 2.58 \cdot \text{RMS}_{\text{prev}} \tag{38}$$

The threshold 2.58 corresponds to the 99% quantile of a standard normal distribution, meaning approximately 1% of points are rejected as outliers.

7.9 Convergence Behavior

Point-to-plane ICP typically converges faster than point-to-point ICP because:

1. It correctly handles the case where surfaces slide tangentially
2. The error function has a flatter minimum in tangent directions
3. Convergence is linear in the tangent plane vs. quadratic for point-to-point

For well-behaved 2.5D data, β and γ should remain small (< 1). Large values may indicate:

- Surface tilt between measurements
- Mounting differences
- Algorithm divergence (reduce iterations or check input data)

8 Difference Image Calculation

After registration, the difference image shows the per-pixel height difference between the aligned data and model:

$$\Delta z(r, c) = z_{\text{data}}^{\text{transformed}}(r, c) - z_{\text{model}}(r, c) \tag{39}$$

For each valid model pixel (r_m, c_m):

1. Apply inverse transform to find corresponding data position
2. Use bilinear interpolation to get data z-value
3. Compute difference: $\Delta z = z_d + t_z - z_m$

Pixels are marked as invalid if:

- The transformed position falls outside the data image bounds
- Any of the four interpolation corners are invalid (holes)
- The pixel is outside the ROI

9 Why Not Standard 3D ICP?

Standard ICP fails for 2.5D data due to:

9.1 Scale Imbalance

With $z \approx 1000$ and $x, y \approx 0.01$, the z-axis dominates any 3D distance metric:

$$d_{3D} = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + (z_1 - z_2)^2} \approx |z_1 - z_2| \tag{40}$$

9.2 Incorrect Correspondences

3D nearest-neighbor finds points that are close in the z-direction, not points that correspond to the same physical location on the surface.

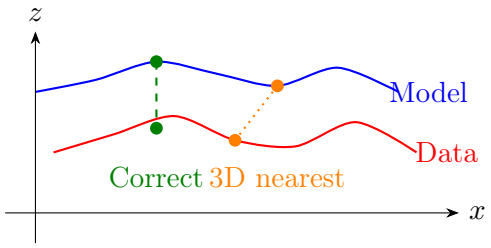

Figure 5: Correspondence problem: 3D nearest-neighbor (orange) finds wrong correspondences. The correct correspondence (green) is based on (x, y) position.

10 References

1. Kanatani, K. (1994). "Analysis of 3-D Rotation Fitting." *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 16(5), 543–549.

2. Horn, B.K.P. (1987). "Closed-form solution of absolute orientation using unit quaternions." *Journal of the Optical Society of America A*, 4(4), 629–642.

3. Besl, P.J. and McKay, N.D. (1992). "A Method for Registration of 3-D Shapes." *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 14(2), 239–256.

4. Eggert, D.W., Lorusso, A., and Fisher, R.B. (1997). "Estimating 3-D rigid body transformations: a comparison of four major algorithms." *Machine Vision and Applications*, 9(5-6), 272–290.

5. Neugebauer, P.J. (1997). "Geometrical cloning of 3D objects via simultaneous registration of multiple range images." *Proceedings of the International Conference on Shape Modeling and Applications*, 130–139. (Point-to-plane ICP formulation used in the 6-DOF Refine algorithm.)

6. Chen, Y. and Medioni, G. (1992). "Object modelling by registration of multiple range images." *Image and Vision Computing*, 10(3), 145–155. (Original point-to-plane ICP concept.)

7. Low, K.-L. (2004). "Linear least-squares optimization for point-to-plane ICP surface registration." *Technical Report TR04-004, University of North Carolina at Chapel Hill.* (Linearization derivation.)